

LAPORAN PRATIKUM STRUKTUR DATA

ADT (ABSTRACT DATA TYPE) – IMPLEMENTASI KELAS JAM

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : Dr.WAHYUDI, S.T, M.T

ASISTEN LABORATORIUM : Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan izin-Nya laporan praktikum “**ADT (Abstract Data Type) – Implementasi Kelas Jam**” ini dapat diselesaikan dengan baik. Saya ucapkan terima kasih sebesar-besarnya kepada Dr. Wahyudi, S.T, M.T selaku dosen pengampu mata kuliah Struktur Data, serta kepada Uda Aufan Taufiqurrahman selaku asisten laboratorium yang telah membimbing pelaksanaan praktikum ini.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep *Abstract Data Type* (ADT) dalam bahasa pemrograman Java, khususnya melalui implementasi kelas Jam. Saya menyadari bahwa pemahaman tentang ADT merupakan fondasi penting dalam pengembangan perangkat lunak yang terstruktur dan mudah dikelola.

Saya menyadari bahwa penulisan laporan praktikum ini masih jauh dari kata sempurna baik dari segi pembahasan dan penulisannya. namun dengan demikian saya telah berupaya dengan segala kemampuan dan pengetahuan yang dimiliki supaya laporan ini dapat selesai dengan baik dan oleh karenanya saya dengan rendah hati menerima saran masukan dan kritikan guna penyempurnaan laporan ini.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	3
1.1 Latar Belakang	3
1.2 Tujuan.....	3
1.3 Manfaat	3
BAB II PEMBAHASAN	4
2.1 Perulangan Pada Java	4
2.2 Langkah pengerjaan	4
2.3 Latihan	10
BAB III KESIMPULAN	13
3.1 Kesimpulan	13
3.2 Saran.....	13
DAFTAR PUSTAKA	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Abstract Data Type (ADT) adalah suatu tipe data yang didefinisikan oleh perilaku (operasi) dari sudut pandang pengguna data, bukan dari implementasi teknisnya. ADT memisahkan antarmuka (*interface*) dari implementasi konkritnya, sehingga memungkinkan pengembangan program yang lebih modular dan mudah dirawat.

Salah satu contoh sederhana dari ADT adalah kelas Jam yang merepresentasikan waktu dalam jam, menit, dan detik. Kelas ini menyediakan operasi-operasi seperti konversi ke detik, penjumlahan dua waktu, perbandingan, serta validasi nilai. Dalam praktikum ini, peserta akan mengimplementasikan ADT Jam lengkap dengan *driver* untuk menguji fungsionalitasnya.

1.2 Tujuan

1. Menambah pemahaman tentang enkapsulasi dan pembentukan tipe data baru di Java.
2. Melatih keterampilan dalam menulis kode yang terstruktur, mudah dibaca, dan dapat digunakan ulang.
3. Menyediakan bekal awal untuk mempelajari struktur data lebih lanjut seperti ArrayList, Stack, dan Queue.

1.3 Manfaat

1. Menambah pemahaman tentang struktur perulangan *while* dalam bahasa Java.
2. Menambah wawasan serta bekal mahasiswa mengenai pemrograman

.

BAB II

PEMBAHASAN

2.1 ADT Pada java

ADT adalah suatu tipe data yang didefinisikan oleh kumpulan operasi yang sah terhadap data tersebut. Dalam Java, ADT diimplementasikan menggunakan *class* dengan atribut bersifat *private* dan metode-metode *public* sebagai antarmuka.

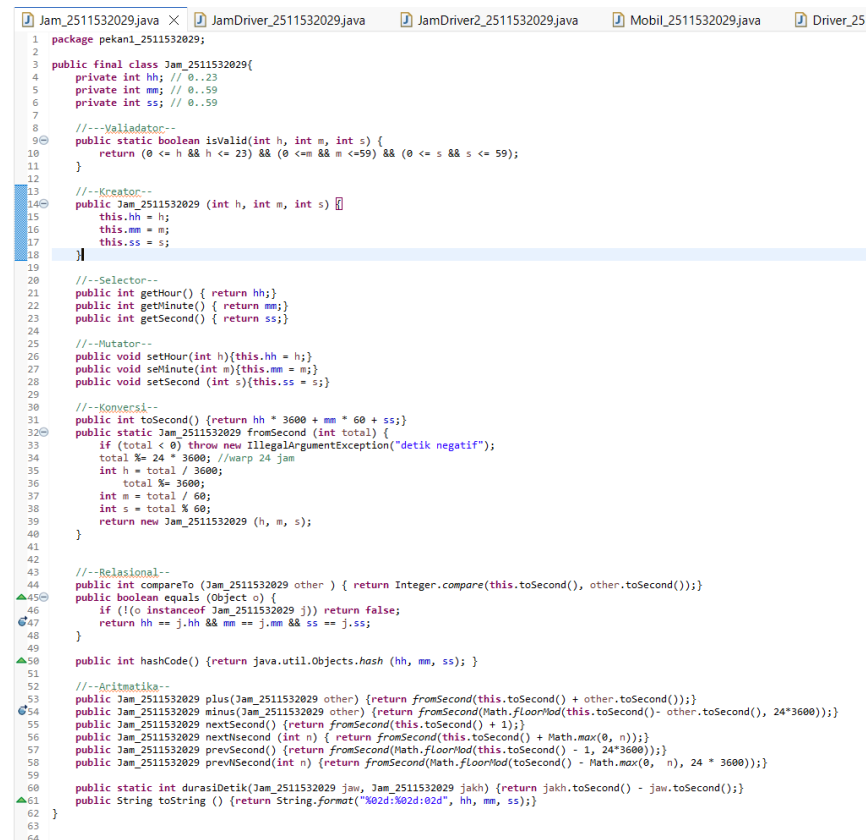
- Karakteristik ADT:
 - Enkapsulasi – data disembunyikan dari pengguna.
 - Operasi yang jelas – pengguna hanya berinteraksi melalui metode yang disediakan.
 - Independensi implementasi – perubahan internal tidak memengaruhi pengguna selama antarmuka tetap.
- Kelas Jam yang akan dibuat memiliki atribut:
 - hh (jam,0-23)
 - mm (menit,0-59)
 - ss (detik,0-59)

2.2 Langkah pengerjaan

1. Pembuatan kelas Jam

Kelas Jam dirancang sebagai ADT dengan atribut hh, mm, ss yang bersifat private. Semua operasi terhadap jam diimplementasikan melalui metode publik. Berikut adalah implementasi lengkap yang menggabungkan keempat aspek: deklarasi & validasi, konstruktor & selektor/mutator, konversi, serta operasi aritmatika dan relasional. Eksekusi Awal Do-Block

Program:



```
1 package pekan1_2511532029;
2
3 public final class Jam_2511532029 {
4     private int hh; // 0..23
5     private int mm; // 0..59
6     private int ss; // 0..59
7
8     //--Validator--
9     public static boolean isValid(int h, int m, int s) {
10         return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
11     }
12
13     //--Kreator--
14     public Jam_2511532029 (int h, int m, int s) {
15         this.hh = h;
16         this.mm = m;
17         this.ss = s;
18     }
19
20     //--Selector--
21     public int getHour() { return hh; }
22     public int getMinute() { return mm; }
23     public int getSecond() { return ss; }
24
25     //--Mutator--
26     public void setHour(int h){this.hh = h;}
27     public void setMinute(int m){this.mm = m;}
28     public void setSecond (int s){this.ss = s;}
29
30     //--Konversi--
31     public int toSecond() {return hh * 3600 + mm * 60 + ss;}
32     public static Jam_2511532029 fromSecond (int total) {
33         if (total < 0) throw new IllegalArgumentException("detik negatif");
34         total %= 24 * 3600; //wrap 24 jam
35         int h = total / 3600;
36         total %= 3600;
37         int m = total / 60;
38         int s = total % 60;
39         return new Jam_2511532029 (h, m, s);
40     }
41
42     //--Relasional--
43     public int compareTo (Jam_2511532029 other) { return Integer.compare(this.toSecond(), other.toSecond()); }
44     public boolean equals (Object o) {
45         if (!(o instanceof Jam_2511532029)) return false;
46         return hh == j.hh && mm == j.mm && ss == j.ss;
47     }
48
49     public int hashCode() {return java.util.Objects.hash (hh, mm, ss); }
50
51     //--Aritmatika--
52     public Jam_2511532029 plus(Jam_2511532029 other) {return fromSecond(this.toSecond() + other.toSecond());}
53     public Jam_2511532029 minus(Jam_2511532029 other) {return fromSecond(Math.floorMod(this.toSecond() - other.toSecond(), 24*3600));}
54     public Jam_2511532029 nextSecond() {return fromSecond(this.toSecond() + 1);}
55     public Jam_2511532029 nextSecond (int n) { return fromSecond(this.toSecond() + Math.max(0, n));}
56     public Jam_2511532029 prevSecond() {return fromSecond(Math.floorMod(this.toSecond() - 1, 24*3600));}
57     public Jam_2511532029 prevSecond(int n) {return fromSecond(Math.floorMod(toSecond() - Math.max(0, n), 24 * 3600));}
58
59     public static int durasiDetik(Jam_2511532029 jaw, Jam_2511532029 jakh) {return jakh.toSecond() - jaw.toSecond();}
60     public String toString () {return String.format("%02d:%02d:%02d", hh, mm, ss);}
61 }
62
63
64
```

Gambar 2.1

Penjelasan program

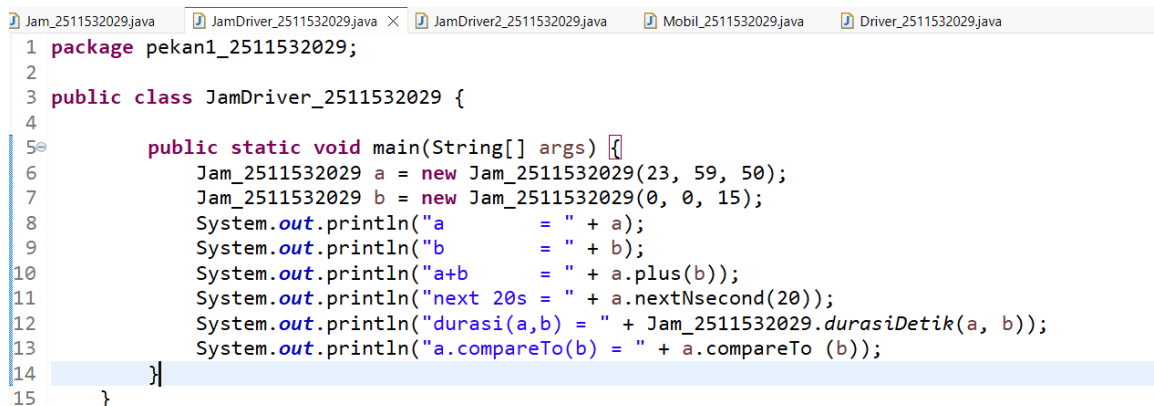
- (a) Validator – isValid() memastikan jam, menit, detik dalam rentang yang sah.
- (b) Konstruktor, selektor, mutator – digunakan untuk membuat objek dan mengakses/mengubah atribut secara terkontrol.
- (c) Konversi – toSeconds() mengubah jam menjadi total detik; fromSeconds() membangun objek Jam dari total detik dengan wrap around 24 jam.
- (d) Operasi aritmatika & relasional –

plus(), nextSecond(), prevSecond(), compareTo(), dan durasiDetik() menyediakan fungsionalitas penjumlahan, pergeseran waktu, perbandingan, dan selisih antar dua jam.

2. Pembuatan Jam Driver

Driver ini menguji beberapa operasi dasar tanpa input dari pengguna.

Program:



```
1 package pekan1_2511532029;
2
3 public class JamDriver_2511532029 {
4
5     public static void main(String[] args) {
6         Jam_2511532029 a = new Jam_2511532029(23, 59, 50);
7         Jam_2511532029 b = new Jam_2511532029(0, 0, 15);
8         System.out.println("a = " + a);
9         System.out.println("b = " + b);
10        System.out.println("a+b = " + a.plus(b));
11        System.out.println("next 20s = " + a.nextNsecond(20));
12        System.out.println("durasi(a,b) = " + Jam_2511532029.durasiDetik(a, b));
13        System.out.println("a.compareTo(b) = " + a.compareTo (b));
14    }
15 }
```

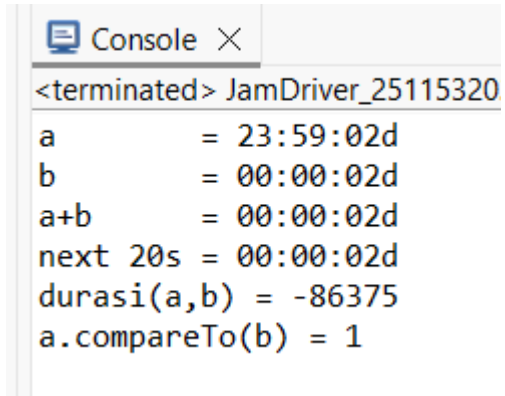
Gambar 2.2

Penjelasan Program:

1. Jam a = new Jam(23, 59, 50);
Membuat objek a dengan waktu pukul 23:59:50. Konstruktor akan mengisi atribut hh=23, mm=59, ss=50.
2. Jam b = new Jam(0, 0, 15);
Membuat objek b dengan waktu pukul 00:00:15.
3. System.out.println("a = " + a);
Mencetak objek a. Karena kelas Jam telah meng-override metode toString() (berupa format hh:mm:ss), maka outputnya adalah a = 23:59:50.
4. System.out.println("a+b = " + a.plus(b));
Memanggil metode plus(b) yang mengembalikan objek Jam baru hasil penjumlahan a dan b.
 - o a.toSeconds() = $23 \times 3600 + 59 \times 60 + 50 = 86390$ detik.
 - o b.toSeconds() = 15 detik.
 - o Total = 86405 detik.
 - o fromSeconds(86405) akan melakukan wrap 24 jam: $86405 \% 86400 = 5$ detik.
 - o Hasilnya adalah 00:00:05.
 - o Output: a+b = 00:00:05.
5. System.out.println("next 20s = " + a.nextNSeconds(20));
nextNSeconds(20) adalah metode (tidak ditampilkan di potongan awal, tetapi dapat diimplementasi) yang menambahkan 20 detik ke a. Dari 23:59:50 ditambah 20 detik menjadi 00:00:10 (karena melewati tengah malam). Output: next 20s = 00:00:10.
6. System.out.println("durasi(a,b) = " + Jam.durasiDetik(a, b));
Metode statis durasiDetik menghitung selisih absolut dalam detik antara a dan b.

- `a.toSeconds() = 86390`, `b.toSeconds() = 15`.
 - Selisih = 86375 detik.
 - Output: `durasi(a,b) = 86375`.
7. `System.out.println("a.compareTo(b) = " + a.compareTo(b));`
`compareTo` mengembalikan selisih detik (positif jika a lebih besar dari b). Nilai kembalian = $86390 - 15 = 86375$. Output: `a.compareTo(b) = 86375`.

Output :



```
<terminated> JamDriver_25115320
a          = 23:59:02d
b          = 00:00:02d
a+b        = 00:00:02d
next 20s   = 00:00:02d
durasi(a,b) = -86375
a.compareTo(b) = 1
```

Gambar 2.3

3. Pembuatan Driver Interaktif

Driver ini memungkinkan pengguna memasukkan dua buah waktu, disertai validasi input menggunakan perulangan while(true) sampai data yang dimasukkan benar

Program:

```
Jam_2511532029.java JamDriver_2511532029.java *JamDriver2_2511532029.java Mobil_2511532029.java Driver_2511532029.java
1 package pekan1_2511532029;
2 import java.util.Scanner;
3
4 public class JamDriver2_2511532029 {
5     public static void main(String[] args) {
6         Scanner input = new Scanner(System.in);
7         System.out.println("=== Program Driver Objek Jam ===");
8
9         // 1. Input Jam Pertama
10        System.out.println("\n--- Input Jam 1 ---");
11        Jam_2511532029 j1 = buatJamDariInput(input);
12
13        // 2. Input Jam Kedua
14        System.out.println("\n--- Input Jam 2 ---");
15        Jam_2511532029 j2 = buatJamDariInput(input);
16
17        // 3. Menampilkan Data
18        System.out.println("\n--- Hasil Operasi ---");
19        System.out.println("Jam 1 (String) : " + j1.toString());
20        System.out.println("Jam 2 (String) : " + j2.toString());
21        System.out.println("Jam 1 dalam detik : " + j1.toSecond());
22        System.out.println("Jam 2 dalam detik : " + j2.toSecond());
23
24        // 4. Operasi Relasional (Perbandingan)
25        int perbandingan = j1.compareTo(j2);
26        if (perbandingan > 0) {
27            System.out.println("Status : Jam 1 lebih lambat (setelah) jam 2");
28        } else if (perbandingan < 0) {
29            System.out.println("Status : Jam 1 lebih awal (sebelum) Jam 2");
30        } else {
31            System.out.println("Status : Jam 1 dan Jam 2 sama persis");
32        }
33
34        // 5. Operasi aritmatika
35        System.out.println("Durasi (j1 ke j2) : " + Jam_2511532029.durasiDetik(j1, j2) + " detik");
36
37        Jam_2511532029 jNext = j1.nextSecond();
38        System.out.println("Jam 1 Detik berikutnya : " + jNext);
39
40        Jam_2511532029 jPrev = j1.prevSecond();
41        System.out.println("Jam 1 Detik sebelumnya : " + jPrev);
42
43        // 6. Operasi Penjumlahan Jam
44        Jam_2511532029 jHasilPlus = j1.plus(j2);
45        System.out.println("Hasil j1 + j2 : " + jHasilPlus);
46
47        input.close();
48        System.out.println("\nProgram Selesai.");
49        System.out.println("\nProgram Selesai.");
50    }
51
52    /**
53     * Prosedur pembantu untuk melakukan input dan validasi.
54     * Mengulang input jika angka di luar jangkauan (0-23 atau 0-59).
55     */
56    private static Jam_2511532029 buatJamDariInput(Scanner sc) {
57        int h, m, s;
58        while (true) {
59            System.out.print("Masukkan jam (0-23) : ");
60            h = sc.nextInt();
61            System.out.print("Masukkan menit (0-59) : ");
62            m = sc.nextInt();
63            System.out.print("Masukkan detik (0-59) : ");
64            s = sc.nextInt();
65
66            // Validasi input
67            if (h >= 0 && h <= 23 && m >= 0 && m <= 59 && s >= 0 && s <= 59) {
68                return new Jam_2511532029(h, m, s); // Mengembalikan objek jika valid
69            } else {
70                System.out.println("[Error] Input tidak valid! Silakan ulangi.");
71            }
72        }
73    }
74 }
```

Gambar 2.3

Penjelasan :

a. Metode main

- `Scanner input = new Scanner(System.in);`
Objek Scanner digunakan untuk membaca ketikan dari keyboard.
- `Jam j1 = buatJamDariInput(input);`
Memanggil metode pembantu untuk membuat objek Jam pertama. Metode ini akan meminta input berulang sampai valid.
- `Jam j2 = buatJamDariInput(input);`
Proses yang sama untuk jam kedua.
- **Operasi setelah input**
Menampilkan kedua jam, selisih durasi, detik berikutnya dari jam pertama, dan hasil penjumlahan. Semua output memanfaatkan metode yang telah didefinisikan di kelas Jam.
- `input.close();`
Menutup Scanner untuk mencegah kebocoran sumber daya.

b. Metode `buatJamDariInput(Scanner sc)`

Metode ini adalah jantung dari validasi input. Mari kita bedah:

- `while (true)` – Perulangan tak terbatas (infinite loop) yang hanya akan dihentikan jika input valid.
- Di dalam perulangan:
 - Meminta pengguna memasukkan jam, menit, dan detik.
 - `if (Jam.isValid(h, m, s))` – Memeriksa apakah ketiga angka berada dalam rentang yang benar.
 - Jika valid → `return new Jam(h, m, s);` – perulangan berhenti dan objek Jam dikembalikan.
 - Jika tidak valid → mencetak pesan error, lalu perulangan berulang kembali ke awal (tanpa mengembalikan objek). Pengguna diminta mengulangi input dari awal.

Output:

```
Console X
<terminated> JamDriver2_2511532029 [Java Application] C:\Users\USER\p2\por
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan jam (0-23) : 22
Masukkan menit (0-59) : 33
Masukkan detik (0-59) : 44

--- Input Jam 2 ---
Masukkan jam (0-23) : 11
Masukkan menit (0-59) : 22
Masukkan detik (0-59) : 33

--- Hasil Operasi ---
Jam 1 (String) : 22:33:02d
Jam 2 (String) : 11:22:02d
Jam 1 dalam detik : 81224
Jam 2 dalam detik : 40953
Status : Jam 1 lebih lambat (setelah) jam 2
Durasi (j1 ke j2) : -40271 detik
Jam 1 Detik berikutnya: 22:33:02d
Jam 1 Detik sebelumnya: 22:33:02d
Hasil j1 + j2 : 09:56:02d

Program Selesai.
```

Gambar 2.4

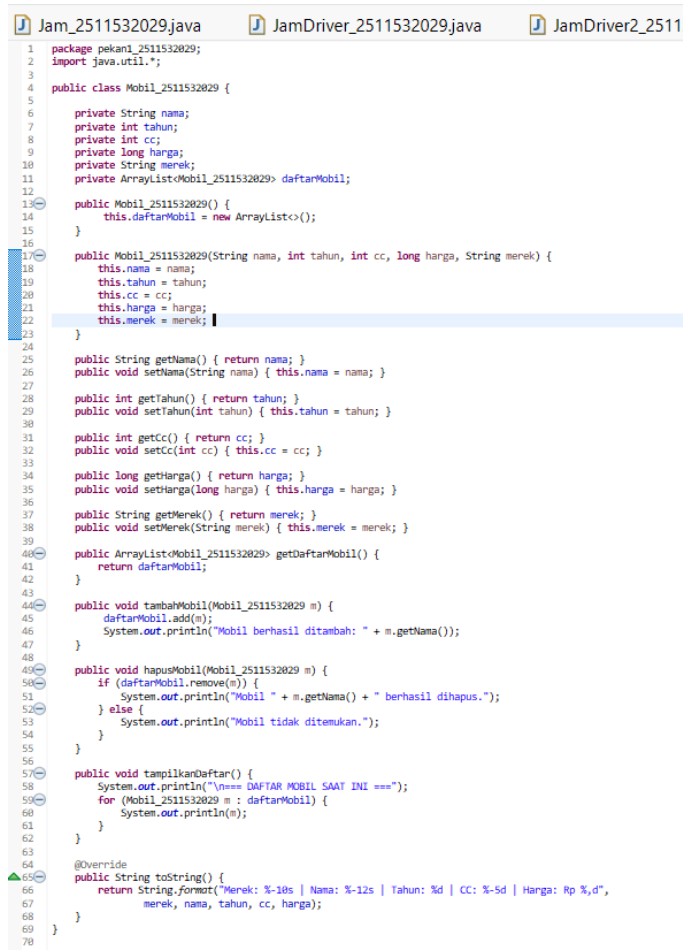
2.3 Latihan

Deskripsi Program:

Program ini mendemonstrasikan pembuatan ADT Mobil yang memiliki atribut seperti nama, tahun, cc, harga, dan merek. Selain itu, terdapat kelas Mobil_2511532029 yang juga berfungsi sebagai manager (mengandung ArrayList<Mobil_2511532029>) untuk menyimpan, menambah, menghapus, dan menampilkan daftar mobil.

Kelas Driver_2511532029 berfungsi sebagai penguji (driver) yang membuat beberapa objek mobil, menambahkannya ke dalam daftar, menampilkan daftar, lalu menghapus satu mobil.

1.Program Mobil_2511532029



```
1 package pekan1_2511532029;
2 import java.util.*;
3
4 public class Mobil_2511532029 {
5
6     private String nama;
7     private int tahun;
8     private int cc;
9     private long harga;
10    private String merek;
11    private ArrayList<Mobil_2511532029> daftarMobil;
12
13    public Mobil_2511532029() {
14        this.daftarMobil = new ArrayList<>();
15    }
16
17    public Mobil_2511532029(String nama, int tahun, int cc, long harga, String merek) {
18        this.nama = nama;
19        this.tahun = tahun;
20        this.cc = cc;
21        this.harga = harga;
22        this.merek = merek;
23    }
24
25    public String getNama() { return nama; }
26    public void setNama(String nama) { this.nama = nama; }
27
28    public int getTahun() { return tahun; }
29    public void setTahun(int tahun) { this.tahun = tahun; }
30
31    public int getCc() { return cc; }
32    public void setCc(int cc) { this.cc = cc; }
33
34    public long getHarga() { return harga; }
35    public void setHarga(long harga) { this.harga = harga; }
36
37    public String getMerek() { return merek; }
38    public void setMerek(String merek) { this.merek = merek; }
39
40    public ArrayList<Mobil_2511532029> getDaftarMobil() {
41        return daftarMobil;
42    }
43
44    public void tambahMobil(Mobil_2511532029 m) {
45        daftarMobil.add(m);
46        System.out.println("Mobil berhasil ditambah: " + m.getNama());
47    }
48
49    public void hapusMobil(Mobil_2511532029 m) {
50        if (daftarMobil.remove(m)) {
51            System.out.println("Mobil " + m.getNama() + " berhasil dihapus.");
52        } else {
53            System.out.println("Mobil tidak ditemukan.");
54        }
55    }
56
57    public void tampilkanDaftar() {
58        System.out.println("\n===== DAFTAR MOBIL SAAT INI =====");
59        for (Mobil_2511532029 m : daftarMobil) {
60            System.out.println(m);
61        }
62    }
63
64    @Override
65    public String toString() {
66        return String.format("Merek: %-10s | Nama: %-12s | Tahun: %d | CC: %-5d | Harga: Rp %d",
67            merek, nama, tahun, cc, harga);
68    }
69
70 }
```

Gambar 2.5

Struktur Kelas Mobil

a) Atribut (private)

java

private String nama;

private int tahun;

private int cc;

private long harga;

private String merek;

private ArrayList<Mobil_2511532029> daftarMobil;

- Atribut nama, tahun, cc, harga, merek merepresentasikan data sebuah mobil.
- Atribut daftarMobil adalah ArrayList yang menyimpan kumpulan objek Mobil_2511532029. Dengan demikian, kelas ini memiliki **dua peran**: sebagai representasi mobil tunggal dan sebagai kumpulan mobil.

b) Konstruktor

1. Konstruktor tanpa parameter

public Mobil_2511532029() → hanya menginisialisasi daftarMobil = new ArrayList<>();. Digunakan untuk membuat objek manager sebelum menambah mobil.

2. Konstruktor dengan parameter

public Mobil_2511532029(String nama, int tahun, int cc, long harga, String merek) → mengisi atribut mobil individual.

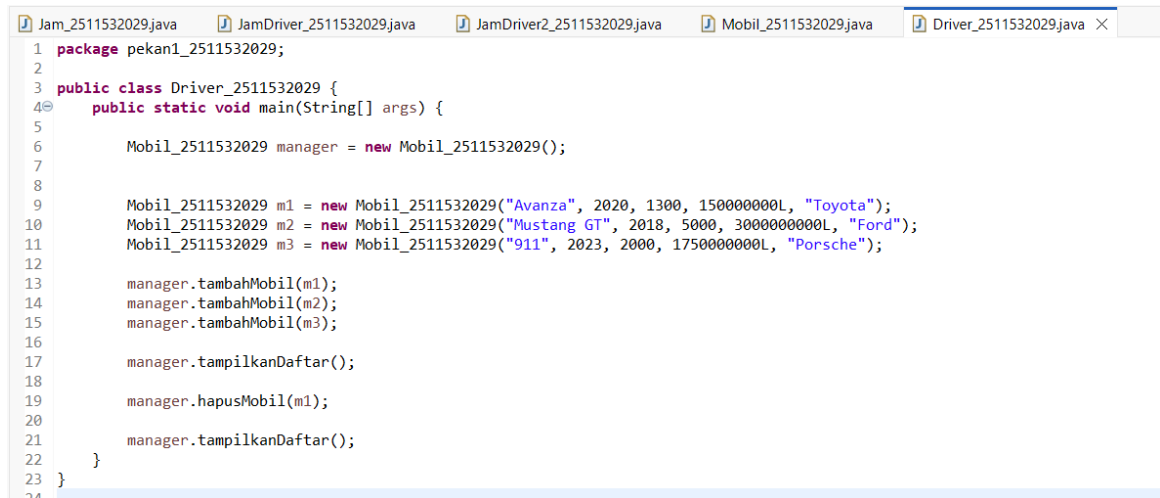
c) Metode Selektor dan Mutator (Getter/Setter)

Disediakan untuk setiap atribut (kecuali daftarMobil yang hanya getter). Ini menerapkan prinsip **enkapsulasi**.

d) Metode Operasi pada Koleksi Mobil

- tambahMobil(Mobil_2511532029 m) → menambahkan objek mobil ke dalam ArrayList, lalu mencetak konfirmasi.
- hapusMobil(Mobil_2511532029 m) → menghapus mobil dari daftar. Mengembalikan pesan sukses atau gagal.
- tampilkanDaftar() → melakukan perulangan for-each untuk mencetak semua mobil yang tersimpan.
- toString() → mengembalikan format string yang rapi untuk menampilkan detail mobil

2. Program Driver_2511532029



```
1 package pekan1_2511532029;
2
3 public class Driver_2511532029 {
4     public static void main(String[] args) {
5
6         Mobil_2511532029 manager = new Mobil_2511532029();
7
8
9         Mobil_2511532029 m1 = new Mobil_2511532029("Avanza", 2020, 1300, 1500000000L, "Toyota");
10        Mobil_2511532029 m2 = new Mobil_2511532029("Mustang GT", 2018, 5000, 3000000000L, "Ford");
11        Mobil_2511532029 m3 = new Mobil_2511532029("911", 2023, 2000, 1750000000L, "Porsche");
12
13        manager.tambahMobil(m1);
14        manager.tambahMobil(m2);
15        manager.tambahMobil(m3);
16
17        manager.tampilkanDaftar();
18
19        manager.hapusMobil(m1);
20
21        manager.tampilkanDaftar();
22    }
23 }
```

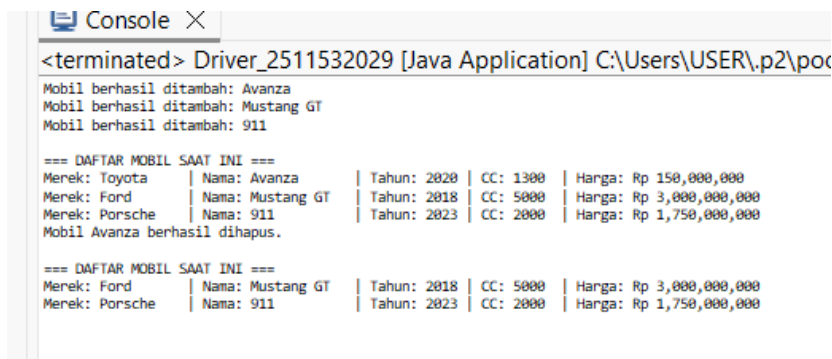
Gambar 2,6

Penjelasan Driver:

```
Mobil_2511532029 manager = new Mobil_2511532029();
Mobil_2511532029 m1 = new Mobil_2511532029("Avanza", 2020, 1300, 1500000000L,
"Toyota");
...
manager.tambahMobil(m1);
manager.tampilkanDaftar();
manager.hapusMobil(m1);
```

- **Objek manager** dibuat dengan konstruktor tanpa parameter. Objek ini tidak merepresentasikan mobil tertentu, tetapi sebagai wadah daftarMobil.
- Tiga objek mobil (m1, m2, m3) dibuat dengan konstruktor berparameter.
- Setiap mobil ditambahkan ke manager menggunakan tambahMobil().
- tampilkanDaftar() mencetak semua mobil.
- hapusMobil(m1) menghapus mobil Avanza dari daftar.
- Terakhir, daftar ditampilkan lagi untuk membuktikan bahwa mobil telah terhapus.

Output:



```
<terminated> Driver_2511532029 [Java Application] C:\Users\USER\.p2\poc
Mobil berhasil ditambah: Avanza
Mobil berhasil ditambah: Mustang GT
Mobil berhasil ditambah: 911

=== DAFTAR MOBIL SAAT INI ===
Merek: Toyota | Nama: Avanza | Tahun: 2020 | CC: 1300 | Harga: Rp 150,000,000
Merek: Ford | Nama: Mustang GT | Tahun: 2018 | CC: 5000 | Harga: Rp 3,000,000,000
Merek: Porsche | Nama: 911 | Tahun: 2023 | CC: 2000 | Harga: Rp 1,750,000,000
Mobil Avanza berhasil dihapus.

=== DAFTAR MOBIL SAAT INI ===
Merek: Ford | Nama: Mustang GT | Tahun: 2018 | CC: 5000 | Harga: Rp 3,000,000,000
Merek: Porsche | Nama: 911 | Tahun: 2023 | CC: 2000 | Harga: Rp 1,750,000,000
```

Gambar 2.7

BAB III KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum implementasi ADT Jam dalam bahasa Java, dapat disimpulkan bahwa Abstract Data Type memungkinkan pemrogram untuk membuat tipe data baru dengan operasi yang jelas dan data yang tersembunyi (enkapsulasi), di mana kelas Jam adalah contoh sederhana namun komprehensif untuk memahami konsep ini. Pemisahan antarmuka dan implementasi tercapai dengan menyediakan metode publik seperti `getHour()`, `plus()`, dan `compareTo()`, sementara atribut `hh`, `mm`, `ss` dibuat `private`.

Penggunaan perulangan sangat penting dalam validasi input, seperti yang ditunjukkan pada `JamDriver2` yang terus meminta input ulang sampai nilai yang dimasukkan valid. Selain itu, operasi aritmatika waktu (penjumlahan, konversi ke detik, dari detik) membutuhkan penanganan `wrap-around` (24 jam) agar hasil tetap berada dalam rentang yang benar. Terakhir, driver program membuktikan bahwa ADT Jam dapat diintegrasikan dengan mudah ke dalam program yang lebih besar, baik yang bersifat interaktif maupun otomatis.

3.2 Saran

1. Akan lebih baik bila dosen menyelenggarakan sesi pra-praktikum di kelas agar mahasiswa dapat memperoleh pemahaman awal yang lebih memadai, sehingga dapat menghindari kepanikan atau kesalahan saat praktikum
2. Sebaiknya dosen membagikan materi praktikum terlebih dahulu melalui iLearn agar mahasiswa bisa mempersiapkan diri sebelum pelaksanaan praktikum

DAFTAR PUSTAKA

- [1] "The while and do-while Statements (The Java™ Tutorials > Learning the Java Language)", Oracle, 2024.
Available: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/while.html>